Modeling in Databricks

After features have been created and stored as feature tables, we create training sets and focus on model training. We will cover modeling in terms of leveraging Databricks to facilitate the model lifecycle in *Chapter 6*. In *Chapter 7*, we'll discuss the Unity Catalog Registry and how to use it to track an enormous amount of information from the associated experiments, in addition to details such as model lineage. You can register multiple versions of a model at every stage and can give these different versions aliases, such as **champion** and **challenger**, or a more specific alias referring to versions A and B in an A/B test. See aliasing in *Figure 2.6*.



Figure 2.6 – A user can alias a model with specific names for A/B testing or multi-armed bandits

In *Chapter 7*, we demonstrate how to trigger a testing script to test every model before having a human review it. Testing models is an efficient practice to reduce the time to production when used consistently and with intention. We suggest defining the criteria for successfully transitioning models/code through isolation environments (from development to stage to production). Clearly defined environments are one of the practices that enable you to create clear and consistent model quality expectations across all models. Be sure to consult *The Big Book of MLOps* on best practices for isolation and model promotion. No matter where your environment is, it is beneficial to incorporate logging into model experimentation.

We discuss logging in the ML context rather than the software development sense. Logging for ML is focused on reproducible research and is also known as experiment tracking. It is common practice to track the following:

- The input data used to train a model
- The parameters used to train a model
- The accuracy and speed performance of a model during training and inference
- The errors that occur during training and inference
- The runtime environment of a model